



KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 3 Quiz

JavaScript'ten TypeScript'e

Doç. Dr. Tolga Berber · <https://avesis.ktu.edu.tr/tberber>

Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

Soru 1 (6 puan)

4 • Değerlendirme Hedef 1

koltukPlaniUret fonksiyonu, koltuklar dizisindeki her koltuk nesnesi için bir tıklama dinleyicisi (*event listener*) oluşturur; kullanıcı bir koltuğa tıkladığında `koltukTiklamaIsleyici`, o koltuğun durum alanını doğrudan değiştirir (`koltuk.durum = ...`). Bu değişikliğin, dışarıdaki (`main.ts`'teki) orijinal koltuklar dizisinde de görünmesinin nedeni hangisidir?

- (a) `koltuk`, `forEach` döngüsünde koltuklar dizisindeki nesnenin kendisine (bir referansa) bağlanır; JavaScript'te nesneler her zaman referans semantiği izler, bu yüzden durum alanını değiştirmek dizideki orijinal nesneyi de değiştirir.
- (b) koltuklar dizisi her tıklamadan sonra otomatik olarak yeniden oluşturulur (kopyalanır); değişiklik bu yeni kopyada görünür hâle gelir.
- (c) durum bir ilkel (*primitive*) değer olduğu için değer semantiği izler, ama TypeScript derleyicisi bunu çalışma zamanında otomatik olarak bir referansa çevirir.
- (d) `addEventListener` çağrısı koltuklar dizisinin ayrı bir kopyasını saklar; değişiklik yalnızca ekrandaki DOM ögesinde görünür, dizinin kendisini etkilemez.

Soru 2 (6 puan)

4 • Değerlendirme Hedef 1

`class Katilimci` söz dizimiyle tanımlanan bir sınıfın metotları hakkında hangisi doğrudur?

- (a) `class`, prototip zincirinden tamamen bağımsız yeni bir bellek modeli kullanır; her `class` metodu, o sınıftan türetilen her nesnenin kendi belleğinde ayrı ayrı saklanır.
- (b) `class`, prototip zincirinin üstüne kurulmuş okunaklı bir sözdizimidir ("syntactic sugar"); `selamla` metodu aslında `Katilimci.prototype` üzerinde durur, sınıftan türetilen her nesne kendi kopyasını taşımaz.
- (c) `class` ile tanımlanan bir metot yalnızca oluşturulduğu örnek (*instance*) üzerinde saklanır; bu yüzden `Object.hasOwn(Katilimci.prototype, "selamla")` her zaman `false` döner.
- (d) `class` sözdizimi yalnızca TypeScript'e özgüdür; düz JavaScript dosyalarında (`.js` uzantılı) `class` anahtar kelimesi kullanılamaz.

Soru 3 (8 puan)**4 • Değerlendirme** Hedef 1

Aşağıdaki kesit `js - temel/closure.js` dosyasından alınmıştır.

```

1 // olusturSayac(), her çağrıldığında kendi bağımsız "deger" değişkenine
2 // sahip YENİ bir fonksiyon döndürür. Dönen iç fonksiyon, dış fonksiyon
3 // (olusturSayac) çoktan çalışmasını bitirmiş olsa bile "deger"e erişmeye
4 // devam eder – bu, closure'ın tanımıdır.
5 function olusturSayac() {
6   let deger = 0;
7   return function () {
8     deger++;
9     return deger;
10  };
11 }
12
13 const sayac1 = olusturSayac();
14 const sayac2 = olusturSayac();
15
16 console.log("sayac1():", sayac1()); // 1
17 console.log("sayac1():", sayac1()); // 2
18 console.log("sayac2():", sayac2()); // 1 – sayac1'den bağımsız kendi deger'i
19 console.log("sayac1():", sayac1()); // 3

```

(a) `sayac2()`'nin ilk çağrısı ne yazdırır ve neden `sayac1()`'in o ana kadarki çağrı sayısından etkilenmez? (b) Kesitin en son satırında `sayac1()` tekrar çağrıldığında hangi değer yazdırılır?

Soru 4 (10 puan)**4 • Değerlendirme** Hedef 1

JavaScript'te ilkel (`number`, `string`, `boolean`) tiplerin değer semantiği, nesne ve dizi tiplerin ise referans semantiği izlediğini biliyorsunuz. Bu farkın, Etkinlik/Bilet Sisteminin koltuklar dizisi üzerinde çalışırken size hangi avantajı sağladığını ve hangi riski taşıdığını kendi cümlelerinizle açıklayınız.

Soru 5 (6 puan)**4 • Değerlendirme** Hedef 2

Her `.js` / `.ts` dosyasının kendi kapsamını koruyup yalnızca `export` edilen isimleri dışarı vermesinin (ES modülü sisteminin) temel amacı nedir?

(a) Node.js'in yalnızca `export` edilen kodu diske kaydetmesini sağlamak; `export` edilmeyen

Soru 5 (devamı)

kod dosyadan tamamen silinir.

- (b) Tarayıcının .js dosyalarını daha hızlı indirmesini sağlamak; export bir sıkıştırma biçimidir.
- (c) Farklı dosyaların aynı ismi kullanmasından doğan isim çakışmalarını önlemek ve bir dosyanın hangi parçalarının başka dosyalara açık olduğunu açıkça belirlemek; export edilmeyen her isim yalnızca o dosyanın kendi kapsamında kalır.
- (d) TypeScript derleyicisinin yalnızca export edilen fonksiyonların tipini kontrol etmesini sağlamak; export edilmeyen kod tip denetiminden tamamen muaftır.

Soru 6 (6 puan)**4 • Değerlendirme Hedef 2**

istemci/package.json'daki "dev": "vite" ile "build": "tsc && vite build" komutları arasındaki fark hakkında hangisi doğrudur?

- (a) İkisi de aynı işlemi yapar; yalnızca terminaldeki çıktı metni farklıdır, kod davranışında hiçbir fark yoktur.
- (b) npm run dev, projeyi tam olarak tip denetiminden geçirip dist/ klasörüne yazar; npm run build ise yalnızca anlık bir önizleme sunucusu başlatır.
- (c) npm run build yalnızca geliştiricinin kendi bilgisayarında test amaçlı çalışır; gerçek kullanıcılar bu komutun ürettiği hiçbir dosyayı görmez.
- (d) npm run dev, kodu hızlı ama tam derlemeden anlık olarak tarayıcıya yansıtan bir geliştirme sunucusu başlatır; npm run build ise tsc && vite build tanımı gereği önce tüm projeyi tip hatalarına karşı denetler, sonra kodu paketleyip dist/ klasörüne üretime hazır statik dosyalar üretir.

Soru 7 (8 puan)**4 • Değerlendirme Hedef 2**

Aşağıdaki iki dosya js -temel/ klasöründedir; ikinci dosya birinciyi içe aktarır.

```
1 // Adlandırılmış (named) export: bu dosyanın kapsamı korunur, yalnızca
2 // export edilen isimler dışarıdan içe aktarılabilir.
3 export function fiyatHesapla(birimFiyat, adet) {
4   return birimFiyat * adet;
5 }
```

```
1 // import ile önceki dosyadan adlandırılmış export'u alıp kullanma.
2 import { fiyatHesapla } from "../modül-disa-aktar.mjs";
3
4 const toplam = fiyatHesapla(150, 3);
5 console.log("toplam:", toplam); // 450
```

- (a) node modul-ice-aktar.mjs komutu çalıştırıldığında terminalde tam olarak ne yazar? (b) İkinci dosyadaki import satırı import { fiyatHesap } from "../modül-disa-aktar.mjs"; (sondaki "la" eksik, yanlış yazılmış) olsaydı ne olurdu?

Soru 8 (10 puan)**4 • Değerlendirme** Hedef 2

package.json'daki "build": "tsc && vite build" tanımında iki komutun && ile birbirine bağlanmasının — yani tsc'nin önce, vite build'in ancak ondan sonra çalışmasının — ne gibi bir güvenlik sağladığını kendi cümlelerinizle açıklayınız.

Soru 9 (6 puan)**4 • Değerlendirme** Hedef 3

Koltuk arayüzünde id ve etkinlikId alanları readonly olarak tanımlanmıştır. Bir Koltuk nesnesi oluşturulduktan sonra k.id = 2; satırını yazarsanız ne olur?

- (a) tsc bunu bir derleme hatası olarak işaretler (TS2540: Cannot assign to 'id' because it is a read-only property); readonly bir alana yalnızca nesne oluşturulurken ilk değer verilebilir, sonradan yeniden atanamaz.
- (b) Kod sorunsuz çalışır, id başarıyla değişir; readonly yalnızca editörde görsel bir uyarı gösterir, derlemeyi asla engellemez.
- (c) readonly yalnızca dizi (number[] gibi) tiplerinde geçerlidir; interface alanlarında hiçbir etkisi yoktur.
- (d) id alanı otomatik olarak eski değerini korur; atama sessizce yok sayılır, ne hata ne uyarı üretilir.

Soru 10 (6 puan)**4 • Değerlendirme** Hedef 3

Liste<T> generic arayüzü Liste<Etkinlik> olarak kullanıldığında hangisi doğrudur?

- (a) T yer tutucusu yalnızca number ya da string gibi ilkel tiplerle kullanılabilir; Etkinlik gibi bir arayüzle generic parametre olarak kullanılamaz.
- (b) T yer tutucusu Etkinlik ile değiştirilir: elemanlar alanı Etkinlik[] olur, ekle bir Etkinlik parametresi alır, bul ise Etkinlik | undefined döndürür — YG I'deki kendi Liste<T> şablon sınıfınızın, kullanım anında somut bir tiplerle çalışması fikriyle birebir aynıdır.
- (c) Liste<T> her kullanıldığında (Liste<Etkinlik>, Liste<Koltuk>) ayrı bir .js dosyası olarak diske derlenir; iki farklı dosya oluşur.
- (d) Generic yalnızca fonksiyon parametrelerinde kullanılabilir; bir interface tanımı içinde T gibi bir tip parametresi tanımlanamaz.

Soru 11 (8 puan)**4 • Değerlendirme** Hedef 3

Etkinlik/Bilet Sistemine, bir katılımcının bir etkinliği değerlendirmesini tutan yeni bir KatilimciDeğerlendirmesi arayüzü eklemek istiyorsunuz. Aşağıdaki alanları taşıyan arayüzü TypeScript ile yazınız: `id` (salt okunur sayı), `etkinlikId` (salt okunur sayı), `yorum` (metin), `puan` — yalnızca 1, 2, 3, 4 ya da 5 tam sayı değerlerinden birini alabilen bir alan (herhangi bir number değil).

Soru 12 (10 puan)**4 • Değerlendirme** Hedef 3

TypeScript'te bir birleşim (*union*) tipteki değişkeni (ör. `KoltukDurumu` tipindeki `koltuk.durum`) kullanmadan önce neden bir tip daraltma (*narrowing*) kontrolüne ihtiyaç duyabilirsiniz? `if (koltuk.durum === 'dolü')` gibi bir kontrolün TypeScript derleyicisine ne sağladığını kendi cümlelerinizle açıklayınız.

Soru 13 (6 puan)**4 • Değerlendirme** Hedef 4

Bilet formunun `submit` olayında `olay.preventDefault()` çağrılmazsa ne olur?

- (a) Kod hiçbir şekilde etkilenmez; `preventDefault()` yalnızca CSS geçiş efektlerini (*transition*) durdurmak için kullanılan kozmetik bir çağrıdır.
- (b) `form.addEventListener('submit', ...)` dinleyicisi hiçbir zaman tetiklenmez; kullanıcı Gönder düğmesine basarsa bile hiçbir kod çalışmaz.
- (c) Formun varsayılan davranışı devreye girer: tarayıcı sayfayı yeniden yükler ve (Hf.6'ya kadar gerçek bir sunucu olmadığı için hedefsiz) bir HTTP isteği göndermeye çalışır; seçim özetini konsola yazan ve `biletOlustur`'u çağırarak TypeScript kodu çalışmadan sayfa yenilenmiş olur — Hafta 2'deki gereksiz sayfa yenilenmesi geri döner.
- (d) tsc derleme hatası verir; `preventDefault()` çağrılmadan bırakılan bir `submit` dinleyicisi `npx tsc --noEmit` adımıyla başarısız kılabilir.

Soru 14 (8 puan)**4 • Değerlendirme** Hedef 4

Aşağıdaki kesit `koltukPlani.ts`'ten alınmıştır.

```
1 // Bir koltuğa tıklandığında durum geçişini uygular: bos -> secili ya da
2 // secili -> bos. dolu koltuklar tıklanamaz (hiçbir şey yapılmaz).
3 function koltukTiklamaIsleyici(
4   koltuk: Koltuk,
5   eleman: HTMLDivElement,
```

Soru 14 (devamı)

```
6  degisiklikOldu: () => void
7  ): void {
8    if (koltuk.durum === 'dolul') {
9      return;
10 }
11
12 koltuk.durum = koltuk.durum === 'bos' ? 'secili' : 'bos';
13 eleman.classList.toggle('secili', koltuk.durum === 'secili');
14 degisiklikOldu();
15 }
```

(a) durum alanı 'dolul' olan bir koltuğa tıklandığında koltukTiklamaIsleyici fonksiyonu ne yapar? (b) durum alanı 'bos' olan bir koltuğa arka arkaya iki kez tıklandığında durum ve elemanın CSS sınıfı hangi sırayla değişir?

Soru 15 (10 puan)**4 • Değerlendirme Hedef 4**

Aşağıdaki kodun çıktısı neden A, D, C, B sırasıyla gerçekleşir? Mikro görev kuyruğu (Promise) ile makro görev kuyruğu (setTimeout) arasındaki önceliği, çağrı yığını kavramına referansla açıklayınız.

```
console.log('A');
setTimeout(() => console.log('B'), 0);
Promise.resolve().then(() => console.log('C'));
console.log('D');
```
